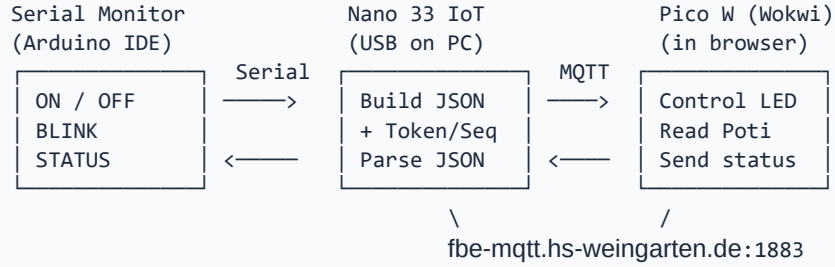
Embedded Lab & Project

Task 3 – MQTT Communication with JSON

This document describes Task 3 of the Embedded Lab & Project course. Task 3 builds directly on the concepts from Task 1 (Hello Embedded World) and Task 2 (UART Communication). The same commands (ON, OFF, BLINK, NOBLINK, STATUS) are now transmitted over the internet via MQTT using JSON messages.

Overview

In Task 3, the local control from Task 2 is extended to bidirectional communication over the internet. A Raspberry Pi Pico W (simulated in Wokwi) and an Arduino Nano 33 IoT (real hardware connected to your PC) communicate via a public MQTT broker (fbe-mqtt.hs-weingarten.de). All messages are formatted as JSON objects.



Property	Details
Duration	2–3 sessions
Environment	Wokwi (Pi Pico W) + Arduino Nano 33 IoT (real hardware)
Prerequisites	Lab Task 2 completed
Libraries	WiFi.h / WiFiNINA.h, PubSubClient, SimpleJson.h (provided)
MQTT Broker	fbe-mqtt.hs-weingarten.de:1883 (no account needed)

From Task 2 to Task 3

The commands in the Serial Monitor are identical to Task 2. The difference lies in the transmission path: instead of local UART, they are now sent as JSON over MQTT via the internet.

Property	Task 2 (UART)	Task 3 (MQTT + JSON)
Connection	Serial Monitor → Pico (direct)	Serial Monitor → Nano → MQTT → Pico
Message format	Plain text strings ("ON", "OFF")	JSON with token + sequence number

Control	Local	Remote over the internet
Security	No protection needed	Token, source check, watchdog, seq nr

Commands

Command	Task 2	Task 3 (new)
ON	LED on (local)	LED on (at Pico in Wokwi)
OFF	LED off (local)	LED off (at Pico in Wokwi)
BLINK	Blink on (local)	Blink on (at Pico in Wokwi)
NOBLINK	Blink off (local)	Blink off (at Pico in Wokwi)
STATUS	Local status	Remote status from Pico
INTERVAL <ms>	–	Set blink interval remotely (50–2000 ms)
POT	–	Re-enable local potentiometer control
STATS	–	Show message statistics
HELP	–	Show available commands

MQTT Topics

Communication uses two topics on the public broker. Each side publishes on one and subscribes to the other.

Topic	Publisher	Subscriber	Content
iem/task3/pico/data	Pico W	Nano IoT	Sensor data (poti, LED, ...)
iem/task3/pico/cmd	Nano IoT	Pico W	Control commands

JSON Message Format

Pico → Nano (sensor data)

```
{
  "token": "iem2026",
  "source": "pico",
  "seq": 42,
  "potValue": 512,
  "blinkEnabled": true,
  "interval": 250,
  "ledState": false,
  "safeState": false,
  "uptime": 120
}
```

Nano → Pico (commands – examples)

```
{"token":"iem2026","source":"nano","seq":0,"blinkEnabled":true}
{"token":"iem2026","source":"nano","seq":1,"blinkEnabled":false,"ledOn":true}
```

```
{"token":"iem2026","source":"nano","seq":2,"interval":200}
{"token":"iem2026","source":"nano","seq":3,"useLocalPot":true}
```

SimpleJson Class

Custom implementation without external libraries (provided as SimpleJson.h). Supports flat JSON objects with String, int, float, and bool types.

```
SimpleJson json;

// Parsing
json.parse("{\"temp\":23.5,\"active\":true}");
float t = json.getFloat("temp");        // 23.5
bool  a = json.getBool("active");        // true

// Building
json.clear();
json.setInt("potValue", 512);
json.setBool("blinkEnabled", true);
char buf[256];
json.toCharArray(buf, sizeof(buf));
// Result: {"potValue":512,"blinkEnabled":true}
```

Robustness Concepts

Since communication runs over a public broker, several protection mechanisms must be implemented:

1. Shared Token

Every message contains the field "token":"iem2026". Messages without a valid token are discarded. This is not cryptographic protection, but a simple filter against random messages on the public broker.

2. Source Validation

The Pico only accepts messages with source="nano", the Nano only those with source="pico". This prevents a device from accidentally processing its own messages.

3. Watchdog / Safe State

Pico: If no valid command has been received for 10 seconds and remote mode is active, the Pico automatically falls back to safe state (local potentiometer control). Nano: If no data arrives from the Pico for 10 seconds, a warning is displayed and the mirror LED is turned off.

4. Sequence Number

Each message contains an incrementing sequence number. Gaps are detected (warning), replays (old/duplicate messages) are discarded.

5. Range Checks

The interval must be between 50 and 2000 ms. Values outside this range are rejected.

Task Steps

Step 0: Wokwi IoT Gateway

The Wokwi simulation needs internet access from your PC to reach the MQTT broker. You must start the Wokwi IoT Gateway application on your desktop before launching the simulation.

- Locate the Wokwi IoT Gateway on your desktop (the exact name will be confirmed by your instructor)
- Start it before running the Wokwi simulation – it bridges WiFi traffic from the browser to your local network
- The gateway must remain running for the entire duration of your session

Step 1: Set up Pico W in Wokwi

1. Create a new Wokwi project: Raspberry Pi Pico W
2. Copy the starter stub (pico_w_mqtt.ino) and SimpleJson.h into the editor
3. Paste diagram.json as the circuit (LED on GP28, button on GP2, poti on GP26)
4. Add the PubSubClient library via the Wokwi Library Manager
5. Start the simulation – the Pico connects to WiFi and MQTT and waits for commands

Step 2: Prepare the Arduino Nano 33 IoT

6. Arduino IDE: Install the board package "Arduino SAMD Boards"
7. Install libraries: WiFiNINA + PubSubClient
8. Copy the starter stub (nano_iot_mqtt.ino) and SimpleJson.h into your sketch folder
9. Update the WiFi credentials (SSID and password in the constants)
10. Upload and open Serial Monitor (115200 baud, "Newline" line ending)

Step 3: Test

Once both sides are running, enter commands in the Serial Monitor:

```
Serial Monitor:                    Wokwi (Pico W):
────────────────                   ────────────────
> BLINK                            LED starts blinking
  -> Blink ON                      (with poti speed)
  ACK:BLINK

> INTERVAL 100                     LED blinks faster
  -> Interval set to 100 ms
  ACK:INTERVAL:100

> STATUS                           (Pico sends its state)
  -> LED: BLINK(100ms)
  -> Poti: 512
  LED:BLINK(100ms)

> OFF                              LED turns off
  -> LED OFF
  ACK:OFF
```

Step 4: Test Robustness – this is optional

Using a separate MQTT client (e.g. mosquitto_pub on the command line), you can inject invalid messages to test the protection mechanisms:

```
# Missing token -> rejected by Pico
mosquitto_pub -h fbe-mqtt.hs-weingarten.de -t "iem/task3/pico/cmd" \
  -m '{"blinkEnabled":false}'

# Wrong token -> rejected
mosquitto_pub -h fbe-mqtt.hs-weingarten.de -t "iem/task3/pico/cmd" \
  -m '{"token":"wrong","source":"nano","seq":0,"blinkEnabled":false}'

# Correct -> accepted
mosquitto_pub -h fbe-mqtt.hs-weingarten.de -t "iem/task3/pico/cmd" \
  -m '{"token":"iem2026","source":"nano","seq":0,"blinkEnabled":false}'
```

✓ Deliverable

Both sides (Pico W in Wokwi + Nano 33 IoT on PC) communicate bidirectionally via MQTT. All commands (ON, OFF, BLINK, NOBLINK, STATUS, INTERVAL, POT) are transmitted and executed correctly. Robustness mechanisms (token, source check, watchdog, sequence number, range checks) are implemented and demonstrated. The onboard LED on the Nano mirrors the Pico LED state. Code is commented.

Starter Stubs

The following code stubs contain the basic structure and all TODOs that students need to implement. The complete SimpleJson.h is provided as a separate file.

Pico W – MQTT Receiver (pico_w_mqtt.ino)

```
// ========================================================================
// Task 3: MQTT + JSON (Pico W, Wokwi)
// Receives control commands via MQTT, publishes sensor data
// ========================================================================

#include <WiFi.h>
#include <PubSubClient.h>
#include "SimpleJson.h"

// -- WiFi (Wokwi) ----------------------------------------------------
const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASS = "";

// -- MQTT ------------------------------------------------------------
const char* MQTT_BROKER  = "fbe-mqtt.hs-weingarten.de";
const int   MQTT_PORT    = 1883;
const char* TOPIC_DATA   = "iem/task3/pico/data";
const char* TOPIC_CMD    = "iem/task3/pico/cmd";

// -- Robustness ------------------------------------------------------
const char* SHARED_TOKEN    = "iem2026";
const char* EXPECTED_SOURCE = "nano";
const unsigned long WATCHDOG_TIMEOUT = 10000;
const int INTERVAL_MIN = 50;
const int INTERVAL_MAX = 2000;

// -- Hardware --------------------------------------------------------
const int LED_PIN    = 28;
const int BUTTON_PIN = 2;
const int POT_PIN    = 26;

// -- State -----------------------------------------------------------
bool blinkEnabled    = true;
bool ledState        = false;
int  overrideInterval = -1;    // -1 = poti controls

unsigned long lastValidCmd = 0;
bool inSafeState = false;
int  expectedSeqNr = -1;
unsigned long seqOutgoing = 0;

// Statistics
unsigned long msgAccepted = 0, msgRejectedJson = 0;
unsigned long msgRejectedAuth = 0, msgSeqGaps = 0;

WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);
SimpleJson jsonOut, jsonIn;

void setupWiFi() {
  // TODO: Connect to WiFi (Wokwi-GUEST, same as Task 1/2)
}

void enterSafeState() {
```

```
  // TODO: Activate safe state
  //   - Set overrideInterval = -1 (back to poti)
  //   - Print warning message
}

void leaveSafeState() {
  // TODO: Leave safe state when valid message arrives
}

bool validateMessage(const SimpleJson& msg) {
  // TODO: Check token (must match SHARED_TOKEN)
  // TODO: Check source (must match EXPECTED_SOURCE)
  return false;
}

bool checkSequence(const SimpleJson& msg) {
  // TODO: Check sequence number
  //   - Detect gaps (warn but accept)
  //   - Detect replays (discard)
  //   - Detect restart (seq == 0 -> resync)
  return true;
}

void processCommand(const SimpleJson& cmd) {
  // TODO: Process received commands:
  //   - blinkEnabled (bool)
  //   - ledOn (bool) -> LED permanently on
  //   - interval (int) -> override with range check
  //   - useLocalPot (bool) -> reset override
}

void mqttCallback(char* topic, byte* payload, unsigned int length) {
  // TODO: Receive and process message:
  //   1. Copy payload into char array (null-terminate!)
  //   2. Parse JSON (jsonIn.parse)
  //   3. Call validateMessage()
  //   4. Call checkSequence()
  //   5. Reset watchdog timer
  //   6. Call processCommand()
}

void mqttReconnect() {
  // TODO: Connect to MQTT + subscribe to TOPIC_CMD
}

void publishSensorData(int potValue, unsigned long blinkInterval) {
  // TODO: Build JSON with SimpleJson:
  //   token, source, seq, potValue, blinkEnabled,
  //   interval, ledState, safeState, uptime
  // Then mqtt.publish(TOPIC_DATA, buf)
}

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  Serial.begin(115200);
  // TODO: setupWiFi()
  // TODO: mqtt.setServer + mqtt.setCallback
}

void loop() {
  // TODO: Check MQTT connection + mqtt.loop()
  // TODO: Check watchdog (enterSafeState if needed)
```

```
  // TODO: Read button (blink toggle, same as Task 1)
  // TODO: Read poti + determine interval
  // TODO: Blink with variable interval (same as Task 1)
  // TODO: Publish sensor data (every 500ms)
}
```

Nano 33 IoT – Serial Command Gateway (nano_iot_mqtt.ino)

```
// ======================================================================
// Task 3: MQTT + JSON - Serial Command Gateway (Nano 33 IoT)
// Receives commands via Serial Monitor, sends them via MQTT to Pico
// ======================================================================

#include <WiFiNINA.h>
#include <PubSubClient.h>
#include "SimpleJson.h"

// -- WiFi (UPDATE THESE!) ------------------------------------------
const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASS = "YOUR_WIFI_PASSWORD";

// -- MQTT ----------------------------------------------------------
const char* MQTT_BROKER  = "fbe-mqtt.hs-weingarten.de";
const int   MQTT_PORT    = 1883;
const char* TOPIC_DATA   = "iem/task3/pico/data";   // Subscribe
const char* TOPIC_CMD    = "iem/task3/pico/cmd";     // Publish

// -- Robustness ----------------------------------------------------
const char* SHARED_TOKEN    = "iem2026";
const char* EXPECTED_SOURCE = "pico";
const unsigned long WATCHDOG_TIMEOUT = 10000;

// -- Last known Pico state -----------------------------------------
int  remotePotValue = 0, remoteInterval = 0, remoteUptime = 0;
bool remoteBlink = false, remoteLedState = false, remoteSafeState = false;

// -- Robustness state ----------------------------------------------
unsigned long lastValidData = 0;
bool picoTimeout = false;
int  expectedSeqNr = -1;
unsigned long seqOutgoing = 0;
unsigned long msgAccepted = 0, msgRejectedJson = 0;
unsigned long msgRejectedAuth = 0, msgSeqGaps = 0;

WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);
SimpleJson jsonOut, jsonIn;

void setupWiFi() {
  // TODO: Connect to WiFi (WiFiNINA)
  //   - Check WiFi.status() == WL_NO_MODULE
  //   - Timeout after 30 attempts
}

bool validateMessage(const SimpleJson& msg) {
  // TODO: Check token + source
  return false;
}

bool checkSequence(const SimpleJson& msg) {
  // TODO: Check sequence number (same logic as Pico)
  return true;
}

void mqttCallback(char* topic, byte* payload, unsigned int length) {
  // TODO: Receive sensor data from Pico:
  //   1. Parse JSON
  //   2. validateMessage()
```

```
  //   3. checkSequence()
  //   4. Store values (remotePotValue etc.)
  //   5. Update mirror LED (LED_BUILTIN)
}

void sendCommand(SimpleJson& cmd) {
  // TODO: Add token, source, seq and publish
  //   cmd.setString("token", SHARED_TOKEN);
  //   cmd.setString("source", "nano");
  //   cmd.setInt("seq", (int)seqOutgoing++);
  //   -> mqtt.publish(TOPIC_CMD, buf)
}

void processSerialInput() {
  // TODO: Read serial input and process commands:
  //   ON       -> blinkEnabled=false, ledOn=true
  //   OFF      -> blinkEnabled=false
  //   BLINK    -> blinkEnabled=true
  //   NOBLINK  -> blinkEnabled=false
  //   INTERVAL <ms> -> interval=<ms> (50..2000)
  //   POT      -> useLocalPot=true
  //   STATUS   -> Display remote state
  //   STATS    -> Display statistics
  //   HELP     -> Show help
}

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(115200);
  while (!Serial) { delay(100); }
  // TODO: setupWiFi()
  // TODO: mqtt.setServer + mqtt.setCallback
}

void loop() {
  // TODO: Check MQTT connection + mqtt.loop()
  // TODO: Check watchdog (warn if Pico not responding)
  // TODO: processSerialInput()
}
```

Hints

- fbe-mqtt.hs-weingarten.de is public – use unique topic paths so groups don’t interfere with each other.
- The shared token is not cryptographic protection, but a simple filter against random messages.
- WATCHDOG_TIMEOUT can be increased to 30s for demos.
- The onboard LED on the Nano mirrors the Pico LED state (visual feedback without extra hardware).
- SimpleJson.h is used identically for both sketches – copy it into each sketch folder.
- Remember to start the Wokwi IoT Gateway on your desktop before launching the simulation.